

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Curiosidades de la información que pueden extraer las personas.....

Sgeun un etsduio de una uivenrsdiad ignlsea, no ipmotra el odren en el que las ltears etsan ersciats, la uicna csoa ipormtnate es que la pmrirea y la utlima ltera esten ecsritas en la psiocion cochrtea. El rsteo peuden estar ttaolmnte mal y aun pordas lerelo sin pobrleams. Etso es pquore no lemeos cada ltera por si msima preo la paalbra es un tdoo.

Pesornamelnte me preace icrneilbe...

ÁREA TEMÁTICA N° 1

PRÁCTICO 1: TEORÍA DE LA INFORMACIÓN

CANTIDAD DE INFORMACIÓN - ENTROPÍA

1. Análisis de Capacidad e Información en Imágenes Digitales

- Suponga una imagen de alta definición (HD) dividida en una matriz de 1920 por 1080 píxeles. Indique la cantidad máxima teórica de información (en bits y en Megabytes) que proporciona este cuadro si está codificada en 24 bits por píxel.
- Luego, considere una imagen en resolución 4K (3840 x 2160 píxeles). ¿Cuánta información máxima se genera si se codifica en formato HDR (High Dynamic Range) utilizando 10 bits por cada canal de color (Rojo, Verde y Azul)?
- Análisis teórico:** La cantidad de información calculada en los incisos anteriores, ¿representa la información "real" (Entropía) de una fotografía o únicamente su capacidad máxima de almacenamiento? Justifique su respuesta explicando el concepto de redundancia espacial.

2. Cantidad de Información en Modelos de Predicción de Texto

Un sistema de autocompletado en un dispositivo móvil debe predecir la siguiente palabra que escribirá un usuario.

- Si el vocabulario total del diccionario del sistema es de 15.000 palabras y, al iniciar una frase, el sistema asume *a priori* que cualquier palabra es equiprobable, ¿cuánta información (en bits) se genera al elegir una palabra exacta?
- Suponga que, analizando el contexto de las palabras anteriores, un modelo de Inteligencia Artificial determina que solo hay 5 palabras lógicas posibles para continuar la frase, siendo estas 5 opciones equiprobables. ¿Cuánta información se genera ahora al confirmar cuál de esas 5 es la palabra correcta?
- ¿Qué cantidad de información (en bits) aportó el contexto procesado por la Inteligencia Artificial al sistema? Explique qué significa este valor en términos de incertidumbre.

3. Propiedades Matemáticas de la Información

- Demostrar las siguientes igualdades correspondientes a la propiedad de cambio de base de los logaritmos:

$$\log_a x = \frac{\log_b x}{\log_b a} = \log_a b \cdot \log_b x$$

- Justificación Teórica:** Indique en qué situaciones prácticas de la Teoría de la Información resulta indispensable utilizar esta propiedad de cambio de base (Pista: Relacione este concepto con las unidades de medida de la información).

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

4. Entropía en Sistemas de Reconocimiento Automático

En el desarrollo de un modelo de Machine Learning para la traducción automatizada, el sistema debe clasificar 5 configuraciones básicas (c_1, c_2, c_3, c_4, c_5). Tras analizar un corpus de datos, se determina que la configuración c_1 es la más frecuente, con una probabilidad $p_1 = 1/3$, mientras que las otras cuatro configuraciones son equiprobables con $p = 1/6$ cada una.

- Calcula la cantidad de información individual que aporta la detección de cada configuración.
- Calcula la entropía total de esta fuente de información.
- Si el sistema se calibra para que todas las configuraciones tengan exactamente la misma probabilidad de aparición, ¿la entropía aumentaría o disminuiría? Justifica tu respuesta.

5. Entropía Máxima vs. Entropía Real en un Pipeline de Datos

Un *pipeline* de backend clasifica las peticiones entrantes de los usuarios en 8 categorías o intenciones distintas.

- Demuestra matemáticamente que si el balanceador de carga distribuye las peticiones de forma tal que las 8 categorías tienen exactamente la misma probabilidad de ocurrir, el sistema alcanza su máxima incertidumbre (entropía máxima). ¿Cuál es ese valor en bits?
- Al pasar el sistema a producción, se observa que el tráfico real está fuertemente sesgado, respondiendo a la siguiente distribución de probabilidades: $p_1 = 0.5$, $p_2 = 0.2$, $p_3 = 0.1$, $p_4 = 0.08$, $p_5 = 0.06$, $p_6 = 0.04$, $p_7 = 0.015$, $p_8 = 0.005$. Calcula la entropía real del sistema en producción.
- Compara ambos resultados (inciso a vs inciso b). ¿Qué nos dice esta diferencia (caída de entropía) sobre la predictibilidad del comportamiento de los usuarios?

6. Entropía Teórica vs. Redundancia en el Lenguaje

Considere un alfabeto teórico de 27 letras.

- Calcule la cantidad de información (Entropía Máxima) que genera la elección de 1 sola letra, una secuencia de 2 letras y una secuencia de 3 letras, asumiendo *a priori* que todas las letras son equiprobables y la elección de cada letra es completamente independiente de las demás.
- Si pasamos de este "alfabeto teórico" al uso real del **idioma español**, el principio de equiprobabilidad desaparece. Explique por qué sucede esto y cómo afecta al valor de la entropía real de un carácter.
- En el idioma español tampoco se cumple el principio de independencia (las letras están condicionadas por las anteriores). Dé un ejemplo de dependencia fuerte entre dos letras en español y explique conceptualmente qué sucede con la cantidad de información generada al leer la segunda letra.

7. Computación Ternaria y Economía de la Base (Radix Economy)

En 1958, la Universidad Estatal de Moscú construyó la computadora "Setun", la cual no utilizaba lógica binaria (bits, base 2), sino lógica ternaria (trits, base 3). Los ingenieros se basaron en el teorema de la "Economía de la Base", el cual postula que el costo de hardware C de un registro es proporcional a la cantidad de dígitos necesarios (I) multiplicada por los niveles de voltaje que debe distinguir cada dígito (la base b). Es decir, $C = I \cdot b$.

Para un sistema que debe poder representar $V = 1.000.000$ de estados diferentes:

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- a) Calcule cuántos "bits" ($b = 2$) y cuántos "trits" ($b = 3$) enteros se necesitan como mínimo. Aplique la fórmula de costo para ambos y demuestre aritméticamente cuál arquitectura es más económica.
- b) Más allá de comparar base 2 y base 3, demuestre analíticamente (utilizando la primera derivada) cuál es la base teórica (número real absoluto) que minimiza la función de costo C para cualquier valor de V .
- c) Si la base 3 es matemáticamente superior, investigue o deduzca por qué la industria global abandonó proyectos como la "Setun" y adoptó el estándar binario.

8. Árboles de Decisión y Capacidad de Información

Se tienen 12 monedas de apariencia idéntica, pero se sabe que exactamente una de ellas es falsa y tiene un peso diferente al resto (no se sabe a priori si es más pesada o más liviana). Usted dispone de una balanza de platillos clásica.

- a) Sabiendo que cada pesada en la balanza (equilibrio, se inclina a la izquierda, se inclina a la derecha) genera una cierta cantidad de información, demuestre matemáticamente por qué son necesarias exactamente 3 pesadas para encontrar la moneda falsa y determinar su anomalía de peso.
- b) Esquematice la metodología (el árbol de decisiones) de la primera pesada y de todas sus ramas para demostrar cómo se divide el espacio de incertidumbre respetando la base ternaria calculada.

9. Información y Reducción de Incertidumbre

En un programa de preguntas, el candidato debe encontrar un premio oculto detrás de una de tres puertas. El premio está detrás de cada puerta con igual probabilidad; una cabra está detrás de las otras dos puertas. Primero, el candidato elige una puerta.

- a) ¿Cuál es la probabilidad de que el premio esté detrás de la puerta elegida?
- b) El presentador del juego sabe detrás de qué puerta está oculto el premio. En lugar de abrir la puerta elegida, abre una de las otras dos puertas que contiene una cabra. Ahora, ofrece al candidato cambiar a la única puerta cerrada que queda. ¿Cuál es la probabilidad de ganar el premio cambiando de puerta?
- c) **Análisis de la Información:** En términos de la Teoría de la Información, el estado inicial del juego posee una incertidumbre máxima. Cuando el presentador abre una puerta con una cabra, está transmitiendo un "mensaje" al candidato. Calcule la Entropía inicial del sistema y explique conceptualmente por qué la acción del presentador inyecta Información al sistema, rompiendo la equiprobabilidad.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

PRÁCTICO 2: CANAL DE INFORMACIÓN

1. Canal de Clasificación y Probabilidades a Posteriori

Un sistema de ruteo automático de tickets de soporte procesa tres tipos de intenciones de entrada (a_1, a_2, a_3). El algoritmo las procesa y emite tres posibles etiquetas de salida para derivar el ticket (b_1, b_2, b_3). Debido a ambigüedades en el texto, el algoritmo comete errores de clasificación, comportándose como un canal ruidoso definido por la siguiente matriz de transición $P(b_j|a_i)$:

	b1	b2	b3
a_1	0,7	0,2	0,1
a_2	0,3	0,4	0,3
a_3	0,5	0,3	0,2

Sabiendo que el tráfico real de la plataforma presenta la siguiente distribución: $P(a_1) = 0.4$, $P(a_2) = 0.3$, $P(a_3) = 0.3$.

- Calcule las probabilidades absolutas de salida $P(b_j)$. ¿Qué porcentaje total de tickets recibe cada etiqueta?
- Calcule la matriz de probabilidades condicionales hacia atrás $P(a_i|b_j)$.
- Si un operador observa que el sistema etiquetó un ticket como b_2 , ¿cuál era la verdadera intención de entrada más probable para ese ticket específico?

2. Canal Binario Asimétrico en Filtros Anti-Spam

Un modelo clasificador de Machine Learning actúa como un filtro anti-spam, comportándose analíticamente como un Canal Binario Simétrico (BSC). La entrada X representa la naturaleza real del correo ($X = 0$ para "Legítimo", $X = 1$ para "Spam") y la salida Y es la decisión del filtro ($Y = 0$ para "Bandeja de Entrada", $Y = 1$ para "Carpeta Spam").

Estadísticamente, la probabilidad de error del canal simétrico es $p = 0.05$.

- Construya la matriz completa de transición del canal $P(Y|X)$ y calcule la matriz de probabilidades conjuntas $P(X, Y)$ asumiendo una probabilidad a priori equiprobable para la fuente.
- Calcule la entropía de la fuente $H(X)$ y el ruido del canal (Equivocación) $H(Y|X)$.
- Obtenga la Información Mutua $I(X; Y)$ de este sistema en producción.
- Calcule la Capacidad del Canal C de ese canal.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

3. Interfaz Determinista sin Ruido

Considere una conexión determinista entre dos microservicios. Existen tres mensajes posibles de entrada (a_1, a_2, a_3) que se mapean de forma exclusiva y biunívoca a tres mensajes de salida (b_1, b_2, b_3).

a) Escriba la matriz de transición de este canal $P(b_j|a_i)$.

b) Proponga teóricamente una distribución de probabilidades para la fuente de entrada que maximice la entropía.

c) Demuestre analíticamente que, en un canal determinista, el ruido es nulo ($H(B|A) = 0$) y la Información Mutua $I(A; B)$ es siempre igual a la entropía de la fuente $H(A)$.

4. Codificación Básica e Información Transmitida

Dada una fuente de información que emite los símbolos (a, b, c) con probabilidades $P(a) = \frac{1}{4}$, $P(b) = \frac{1}{8}$, $P(c) = \frac{5}{8}$, y que atraviesa el siguiente canal de comunicación:

	a'	b'	c'
a	1/4	2/4	1/4
b	1/5	3/5	1/5
c	1/3	1/3	1/3

a) Construya un árbol de codificación binaria válido (código prefijo) para la fuente y asigne el código correspondiente a cada símbolo de entrada (a, b, c).

b) Analice la matriz de transición provista. ¿Se trata de un canal uniforme? Justifique brevemente su respuesta observando las filas y columnas.

c) Calcule la entropía de la fuente $H(A)$ y la Información Mutua $I(A; B)$ para determinar cuánta información realmente logra atravesar el canal sin alterarse por el ruido.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

PRÁCTICO DE MÁQUINA 1

1. Análisis de Información en Señales de Audio (Formato WAV)

El formato WAV sin compresión (PCM) almacena las muestras de audio de forma secuencial. El objetivo de este ejercicio es analizar la cantidad de información real que transporta una señal de audio.

Desarrollar una aplicación de software que analice la distribución estadística de la información y la estructura interna de archivos de audio crudos y comprimidos. Se recomienda utilizar un archivo **.wav** y un archivo **.mp3** que contengan exactamente la misma pista de audio para realizar la comparativa.

a) **Carga y Validación:** Solicitar la ruta de ambos archivos y validar programáticamente que las extensiones y/o los formatos correspondan efectivamente a WAV y MP3.

b) **Análisis de Cabecera (Manipulación de bytes):** Para el archivo WAV, el programa debe leer y aislar la cabecera estándar (RIFF/WAVE) e imprimir por pantalla sus datos principales (por ejemplo: identificadores "RIFF" y "WAVE", tamaño del archivo, frecuencia de muestreo, etc.).

c) **Distribución de Probabilidades:** Leer el contenido de ambos archivos byte por byte (valores del 0 al 255) y calcular la frecuencia relativa de aparición de cada símbolo para obtener su distribución de probabilidad (p_i).

d) **Histogramas:** Generar y mostrar el gráfico del histograma de frecuencias para ambos archivos, permitiendo una comparación visual.

e) **Cálculo de Entropía:** A partir de las probabilidades obtenidas, calcular la entropía empírica de ambas fuentes utilizando la fórmula de Shannon:

$$H = - \sum p_i \log_2(p_i)$$

f) Comparar los valores de entropía obtenidos y la forma de los histogramas. Explicar desde la Teoría de la Información por qué se produce esta diferencia radical entre un archivo de audio sin compresión y uno comprimido.

Formato Wav.

El formato Wave es un subconjunto de la especificación RIFF de Microsoft para formatos multimedia. Los formatos RIFF se caracterizan por una cabecera más una secuencia de bloques de datos.

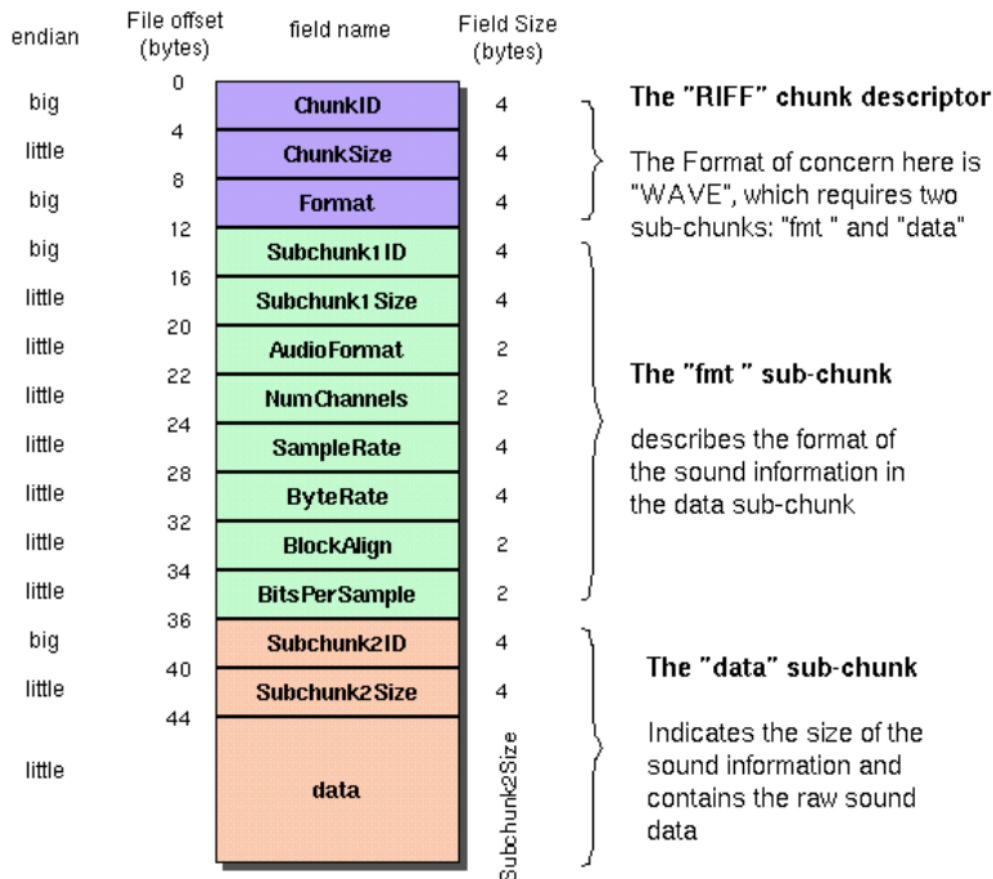
En este sentido el formato Wav divide el archivo en dos bloques, un primer bloque que representa el formato de los datos así como su muestra, y un segundo bloque con los datos propiamente dichos.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

The Canonical WAVE file format



OFFSET	SIZE	NAME	DESCRIPTION
The canonical WAVE format starts with the RIFF header:			
0	4	ChunkID	Contains the letters "RIFF" in ASCII form (0x52494646 big-endian form).
4	4	ChunkSize	36 + SubChunk2Size, or more precisely: 4 + (8 + SubChunk1Size) + (8 + SubChunk2Size) This is the size of the rest of the chunk following this number. This is the size of the entire file in bytes minus 8 bytes for the two fields not included in this count: ChunkID and ChunkSize.
8	4	Format	Contains the letters "WAVE" (0x57415645 big-endian form)
The "WAVE" format consists of two subchunks: "fmt" and "data": The "fmt" subchunk describes the sound data's format:			
12	4	Subchunk1ID	Contains the letters "fmt" (0x666d7420 big-endian form)
16	4	Subchunk1Size	16 for PCM. This is the size of the rest of the Subchunk which follows this number
20	2	AudioFormat	PCM = 1 (i.e. Linear quantization) Values other than 1 indicate some form of compression

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

22	2	NumChannels	Mono = 1, Stereo = 2, etc
24	4	SampleRate	8000, 44100, etc
28	4	ByteRate	$== \text{SampleRate} * \text{NumChannels} * \text{BitsPerSample}/8$
32	2	BlockAlign	$== \text{NumChannels} * \text{BitsPerSample}/8$ The number of bytes for one sample including all channels. I wonder what happens when this number isn't an integer?
34	2	BitsPerSample	8 bits = 8, 16 bits = 16, etc
	2	ExtraParamSize	if PCM, then doesn't exist
	x	ExtraParams	space for extra parameters
The "data" subchunk contains the size of the data and the actual sound:			
36	4	Subchunk2ID	Contains the letters "data" (0x64617461 big-endian form)
40	4	Subchunk2Size	$== \text{NumSamples} * \text{NumChannels} * \text{BitsPerSample}/8$ This is the number of bytes in the data. You can also think of this as the size of the read of the subchunk following this number.
44	*	Data	The actual sound data

2. Análisis de Entropía, Histogramas y Estructura de Archivos (BMP vs. JPG)

Desarrollar una aplicación de software que analice la distribución estadística de la información y la estructura interna de archivos de imagen con y sin compresión espacial. Se recomienda utilizar un archivo **.bmp** y un archivo **.jpg** que contengan exactamente la misma fotografía para realizar la comparativa.

a) Solicitar la ruta de ambos archivos y validar programáticamente que las extensiones y/o los formatos correspondan efectivamente a BMP y JPG.

b) Para el archivo BMP, el programa debe leer y aislar la cabecera estándar (típicamente los primeros 54 bytes) e imprimir por pantalla sus datos principales (por ejemplo: firma del archivo "BM", tamaño del archivo, ancho y alto de la imagen en píxeles, y profundidad de color en bits).

c) Leer el contenido de ambos archivos byte por byte (valores del 0 al 255) y calcular la frecuencia relativa de aparición de cada símbolo para obtener su distribución de probabilidad (p_i).

d) Generar y mostrar el gráfico del histograma de frecuencias para ambos archivos, permitiendo una comparación visual.

e) A partir de las probabilidades obtenidas, calcular la entropía empírica de ambas fuentes utilizando la fórmula de Shannon:

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

$$H = - \sum p_i \log_2(p_i)$$

f) Comparar los valores de entropía obtenidos y la forma de los histogramas. Explicar desde la Teoría de la Información por qué el histograma del archivo BMP presenta picos (evidenciando la redundancia espacial de los píxeles) y por qué el archivo JPG presenta una distribución mucho más uniforme con una entropía cercana a su límite máximo.

Un archivo gráfico en formato BMP (bitmap) tiene la estructura que se ilustra en la siguiente figura:

Cabecera (14 bytes)
Propiedades de la imagen (40 bytes)
Paleta de color (opcional – su tamaño puede variar)
Datos de la imagen

En la cabecera se incluye la siguiente información:

Campo	bytes	Descripción
Signature	2	Siempre es “BM”
FileSize	4	Tamaño del fichero
Reserved	4	No se usa
DataOffset	4	Posición relativa del comienzo de los datos de imagen
En cuanto a la zona de propiedades de la imagen, podemos encontrar la siguiente información:		
Size	4	Tamaño de esta sección (siempre es 40 bytes)
Width	4	Anchura de la imagen en pixels
Height	4	Altura de la imagen en pixels
Planes	2	Número de planos (siempre es 1)
BitCount	2	Bits por píxel (1, 4, 8, 16, 24)
Compression	4	Tipo de compresión empleado (0, 1, 2)
ImageSize	4	Tamaño de la imagen comprimida (=0 si no se comprime)
XPixelsPerM	4	Resolución horizontal: píxeles por metro
YPixelsPerM	4	Resolución vertical: píxeles por metro
ColorsUsed	4	Número de colores usados, si hay paleta.
ColorsImportant	4	Número de colores importantes (=0 si son todos)

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

3. Entropía Empírica en Archivos (Texto vs. Comprimidos) Desarrollar una aplicación de software que lea un archivo arbitrario byte por byte (en un tiempo $O(N)$, siendo N el tamaño del archivo) y calcule su entropía empírica y redundancia.

a) El programa debe calcular la frecuencia relativa de aparición de cada byte (valores del 0 al 255) y calcular entropía:

$$H = - \sum p_i \log_2(p_i)$$

b) Ejecutar el programa utilizando como entrada un archivo de texto puro (.txt) y luego un archivo fuertemente comprimido (.zip o *.rar) de tamaño similar.

c) Explique por qué el valor de la entropía del archivo *.zip se acerca al máximo teórico (8 bits/símbolo) y qué relación conceptual tiene esto con la eliminación de la redundancia en los datos.

4. El índice de coincidencia (IC) en criptografía mide la probabilidad de que dos letras elegidas al azar en un texto sean idénticas. Se calcula con la fórmula , donde es la frecuencia de cada elemento de la fuente y es el total de elementos que tiene la secuencia analizada.

Texto en español: El valor esperado ronda **0.074** debido a la distribución desigual natural de las letras.

Texto completamente aleatorio: El valor esperado es menor, aproximadamente **0.038** (para un alfabeto de 27 letras).

Utilidad principal: Sirve para averiguar la longitud de la clave secreta en cifrados polialfabéticos como *Vigenère*.

Escriba un programa que calcule el IC, de los mismos archivos a los que les calculó la Entropía del punto anterior. Compare los valores de Entropía e IC.

5. Eficiencia de Almacenamiento y Empaquetado a Nivel de Bits (Bitwise) Desarrollar una aplicación que gestione los datos de 20 personas. Por cada persona se debe registrar: Apellido y Nombre, Dirección, DNI, y 8 campos booleanos (Verdadero/Falso) como: estudios primarios, secundarios, universitarios, vivienda propia, obra social, trabaja, etc.

a) Almacenar los datos en un archivo de texto de longitud variable (por ejemplo, formato JSON o CSV), donde los campos booleanos se guarden como cadenas de caracteres ("True", "False", o "S", "N").

b) Almacenar los mismos datos en un archivo binario optimizado de longitud fija. Para los 8 campos booleanos, el programa debe utilizar operadores a nivel de bits (Bitwise) para empaquetar las 8 respuestas en exactamente 1 único byte de memoria por persona.

c) Comparar el tamaño final de ambos archivos en disco (en cantidad de bytes) y redactar una conclusión teórica sobre el impacto de elegir las estructuras de codificación correctas en sistemas de alta escala.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

d) El programa deberá permitir abrir tanto el archivo de **longitud variable** como el de **longitud fija**, leer todos los registros almacenados y mostrar en pantalla los datos recuperados. Para el archivo binario, deberá desempaquetar mediante operaciones **Bitwise** el byte que contiene los 8 campos booleanos.

6. Medición de Distancia entre Cadenas: En la transmisión de datos, es vital detectar qué tan corrompido está un mensaje respecto a su fuente original. Desarrollar un programa que compare dos cadenas de texto y determine su grado de similitud algorítmica.

a) Implementar una función que calcule la "Distancia de Hamming" entre dos cadenas. Demostrar con un ejemplo de ejecución por qué este algoritmo no es aplicable si las cadenas sufren desfases o tienen distinta longitud (ej. "Juan Perez" vs "Jaun Perez").

b) Investigar e implementar el algoritmo de la "Distancia de Levenshtein" (Distancia de Edición), el cual cuenta la cantidad mínima de operaciones matemáticas (inserciones, eliminaciones o sustituciones) necesarias para transformar una cadena en la otra.

c) Probar el programa con nombres mal tipeados (ej. "Horacio López" vs "Oracio López") e imprimir el valor de la distancia (cantidad de errores detectados) calculada por el algoritmo.

d) Proponga un solución, proceso o heurística para la comparación del texto.

7. Detección de Errores: Códigos de Control (Checksum) El sistema de identificación tributaria en Argentina (CUIT/CUIL) utiliza un dígito verificador para detectar alteraciones o errores de tipeo comunes, basándose en el algoritmo de Módulo 11.

a) Investigar la lógica matemática detrás del cálculo del dígito verificador del CUIT/CUIL.

b) Desarrollar un programa que solicite al usuario ingresar por teclado un número de CUIT/CUIL completo de 11 dígitos (como cadena de texto o arreglo numérico).

c) El sistema debe separar los primeros 10 dígitos, aplicar el algoritmo de validación (Módulo 11) para calcular cuál debería ser el dígito de control esperado, y compararlo con el dígito 11 ingresado por el usuario. d) Informar por pantalla si la cadena ingresada es "Válida" o "Inválida", demostrando en el informe cómo esta técnica actúa como un Código de Detección de Errores sistemático.

8. Cálculo de Capacidad de Canal por Búsqueda Exhaustiva (Binario a Cuaternario)

Desarrollar una aplicación de software que determine la Capacidad de Canal (C) de un sistema discreto sin memoria con entrada binaria y salida cuaternaria (2 símbolos de entrada, 4 símbolos de salida). El algoritmo debe ser capaz de resolver tanto canales uniformes como no uniformes.

El programa debe cumplir con los siguientes requerimientos:

a) **Ingreso de la Matriz del Canal:** Solicitar al usuario que ingrese por teclado las probabilidades condicionales hacia adelante $P(Y|X)$. El usuario deberá cargar los 8 valores

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

que conforman la matriz de transición de 2×4 . El programa debe validar matemáticamente que la suma de las probabilidades de cada fila sea estrictamente igual a 1.

b) **Búsqueda Exhaustiva:** Implementar un algoritmo de fuerza bruta mediante bucles para barrer las posibles probabilidades de la fuente de entrada $P(X)$. El incremento (salto) para las probabilidades de entrada debe ser de 0.01 (es decir, evaluando desde 0.00, 0.01, 0.02... hasta 1.00).

c) **Cálculo de Información Mutua:** Para cada distribución de entrada generada por la iteración, el programa debe calcular de forma dinámica:

- Las probabilidades de los símbolos de salida $P(Y)$ utilizando el Teorema de la Probabilidad Total
- La Entropía de la salida $H(Y)$
- La Entropía condicional (Ruido del canal) $H(Y|X)$
- La Información Mutua de esa iteración específica: $I(X;Y) = H(Y) - H(Y|X)$

d) **Maximización:** El programa debe comparar el resultado de $I(X;Y)$ de la iteración actual contra el máximo valor registrado hasta el momento, quedándose siempre con el mayor valor.

e) Una vez finalizada la búsqueda exhaustiva, mostrar por pantalla:

- La Capacidad del Canal (C) encontrada (el máximo valor de $I(X;Y)$ en bits/símbolo).
- La distribución de probabilidades de entrada $P(X=0)$ y $P(X=1)$ exacta que logró maximizar el canal.

9. Simulación y Análisis Teórico de un Canal Binario Simétrico (BSC) Sockets TCP

En este ejercicio, modelaremos un Canal Binario Simétrico (BSC) utilizando una arquitectura Cliente-Servidor mediante Sockets TCP. Un BSC transmite bits (0 o 1) donde cada bit tiene una probabilidad " p " de ser recibido con error, y una probabilidad " $1-p$ " de ser transmitido correctamente. Se provee el script del servidor, el cual actuará como una "caja negra": la probabilidad de error " p " está oculta en el código del servidor y usted deberá descubrirla mediante experimentación.

Fase 1: Transmisión y Tasa de Error Empírica (BER)

1. Desarrolle un script Cliente que se conecte al servidor mediante sockets TCP.
2. Para descubrir el valor de " p ", realice una batería de pruebas empíricas generando tramas sintéticas aleatorias de diferentes magnitudes (por ejemplo: 100, 10.000 y 1.000.000 de bits).
3. Envíe cada trama al servidor. El servidor le devolverá una secuencia de la misma longitud, pero afectada por el ruido del canal.
4. Para cada prueba, compare la trama original con la recibida bit a bit, cuente la cantidad de errores y calcule el BER empírico (Bit Error Rate). En su informe, analice cómo la longitud de la trama enviada afecta la convergencia del BER empírico hacia un valor estable (Ley de los Grandes Números).

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

5. Finalmente, para comprobar el efecto visual del ruido, convierta una frase de texto a binario, pásela por el canal y vuelva a convertir la respuesta a texto para mostrar en pantalla cómo quedó el mensaje alterado.

Fase 2: Modelado Matemático y Capacidad del Canal

Utilizando el valor de la probabilidad de error p teórica configurada en su cliente, y asumiendo que el canal se comporta como un BSC perfecto, el programa debe calcular e imprimir:

1. **La Matriz del Canal:** Muestre las probabilidades de transición $P(y_j|x_i)$.
2. **Probabilidades de la Fuente:** Analice la trama binaria que usted envió originalmente y calcule la probabilidad de entrada de los símbolos $P(x_0)$ y $P(x_1)$ (frecuencia relativa de ceros y unos en su mensaje).
3. **Información Mutua:** Calcule la Información Mutua $I(X;Y)$ de esta transmisión específica, usando las probabilidades calculadas.
4. **Capacidad del Canal:** Calcule la Capacidad máxima C del canal utilizando la entropía condicional:

$$C = 1 - H(p) = 1 - (-p \log_2(p) - (1 - p) \log_2(1 - p))$$

5. **Análisis de Maximización:** Compare el valor obtenido en $I(X;Y)$ con la Capacidad C (aceptando un margen de error mínimo de un 5-10%). Responda en comentarios dentro de su código: *¿Logró su mensaje maximizar la capacidad del canal? Si la respuesta es no, ¿qué característica debería tener la trama de bits enviada para que $I(X;Y)$ sea igual a C ?*

Código del servidor

```
# servidor_bsc.py
#
# Servidor que simula un Canal Binario Simétrico (BSC) sin memoria.
# Utiliza Sockets TCP y un protocolo de enmarcado de longitud (4 bytes).
#
# Python 3.x

import socket
import random
import struct
import threading

# =====
```

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

```
# CONFIGURACION GENERAL
# =====
HOST = "0.0.0.0"
PUERTO = 5555

# Semilla maestra para generar la matriz del canal (fija la
# probabilidad)
SEMILLA_CANAL = 2026

# =====
# GENERACION DEL CANAL (CAJA NEGRA)
# =====
rng_canal = random.Random(SEMILLA_CANAL)
limite_a = rng_canal.random()
limite_b = rng_canal.random()
P_ERROR = min(limite_a, limite_b) * rng_canal.random()

# =====
# PROTOCOLO DE COMUNICACION Y RED
# =====
def recibir_exactamente(sock, cantidad):
    datos = bytearray()
    while len(datos) < cantidad:
        bloque = sock.recv(cantidad - len(datos))
        if not bloque:
            raise ConnectionError("Conexión cerrada por el cliente.")
        datos.extend(bloque)
    return bytes(datos)

def recibir_mensaje(sock):
    encabezado = recibir_exactamente(sock, 4)
    longitud = struct.unpack("!I", encabezado)[0]
    datos = recibir_exactamente(sock, longitud)
    return datos.decode("ascii")

def enviar_mensaje(sock, mensaje):
    datos = mensaje.encode("ascii")
    encabezado = struct.pack("!I", len(datos))
    sock.sendall(encabezado + datos)

# =====
# ATENCION DEL CLIENTE Y RUIDO
# =====
def atender_cliente(cliente, direccion):
    print(f"[+] Cliente conectado: {direccion[0]}:{direccion[1]}")
```

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

```
# Instanciamos el generador de ruido POR CLIENTE.
# Esto asegura que cada sesión nueva sea 100% determinista.
rng_ruido = random.Random(SEMILLA_CANAL + 1)

try:
    while True:
        mensaje = recibir_mensaje(cliente)

        if mensaje == "SALIR":
            break

        if not mensaje:
            enviar_mensaje(cliente, "ERROR: mensaje vacío")
            continue

        if any(bit not in "01" for bit in mensaje):
            enviar_mensaje(cliente, "ERROR: solo se permiten
símbolos 0 y 1")
            continue

        # TRANSMISION A TRAVES DEL CANAL SIMETRICO
        salida = []
        for bit in mensaje:
            # Si el número aleatorio cae dentro de la probabilidad
            # de error, invertimos el bit
            if rng_ruido.random() < P_ERROR:
                salida.append("1" if bit == "0" else "0")
            else:
                salida.append(bit)

        # Devolvemos la salida alterada
        enviar_mensaje(cliente, "".join(salida))

except (ConnectionError, OSError):
    pass
finally:
    cliente.close()
    print(f"[-] Cliente desconectado:
{direccion[0]}:{direccion[1]}")

# =====
# INICIO DEL SERVIDOR
# =====
def iniciar_servidor():
```

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

```
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind((HOST, PUERTO))
servidor.listen()

print("=" * 55)
print(" SERVIDOR - CANAL BINARIO SIMETRICO (BSC)")
print("=" * 55)
print(f"Escuchando en puerto {PUERTO}")
print("La probabilidad de error (p) permanece oculta.")

print("=" * 55)

try:
    while True:
        cliente, direccion = servidor.accept()
        hilo = threading.Thread(
            target=atender_cliente,
            args=(cliente, direccion),
            daemon=True
        )
        hilo.start()
except KeyboardInterrupt:
    print("\nServidor finalizado.")
finally:
    servidor.close()

if __name__ == "__main__":
    iniciar_servidor()
```

Documentación a Entregar:

Se entrega el programa en funcionamiento desarrollado en lenguaje **Python**, y el código fuente debidamente comentado con su README.md con instrucciones para la ejecución.

La entrega se realizará mediante un **repositorio de GitHub por grupo**, el cual deberá contener todos los prácticos de la materia desarrollados hasta el momento. Las correcciones y devoluciones de los trabajos se realizarán a través de las observaciones en los **Issues** del repositorio.

Fecha de entrega: 15 de Septiembre